



Meu Primeiro App com o GitHub Copilot

Guia de prompts passo a passo — app de freelancers com avaliação 360°

Node (backend) + React (frontend) · CRUD, recrutamento, testes, cobertura e documentação

Projeto Social Garapuvu 2026 · Instrutor: Douglas Adriano Queiroz

Este guia é executado ao vivo: você copia cada prompt no chat do GitHub Copilot (VS Code), na ordem apresentada, e mostra o sistema nascendo aos poucos. Ao final você terá uma plataforma de recrutamento de ponta a ponta — com CRUD completo pela interface, uma máquina de status do projeto, edição de perfil, login/logout, testes automatizados em todos os níveis da pirâmide, **cobertura de código de 100%** e testes end-to-end com dois usuários simultâneos.

✓ **O que este guia entrega.** O backend passa **64 testes** (unitários + integração) e o frontend **74 testes** de componente/unidade (React Testing Library) — **cobertura de 100%** (statements, branches, functions e lines) nos dois projetos — além de **6 testes E2E** (Playwright): logout, fluxo completo com dois atores, disputa de duas vagas e seleção com remoção de candidato. Tudo é orquestrado por um Makefile (`make test` , `make test-e2e` , `make test-all` , `make coverage`).

 **A obra inteira, numa olhada (para quem é leigo).** Pense no app como um **prédio**:

- **Backend** (Node + Express) = a **fundação, a estrutura e o encanamento** — fica escondido, mas sustenta tudo e faz a água chegar. É onde moram as regras ("só o dono derruba a parede").
- **Frontend** (React) = o **acabamento visível** — pintura, layout, interruptores. É o que o morador vê e toca.
- **API** = a **portaria/interfone** que liga o morador (frontend) aos serviços do prédio (backend): você aperta um botão, o pedido chega em quem resolve.
- **Banco de dados** = o **cartório/arquivo** onde tudo fica registrado de forma permanente (hoje ainda usamos um "rascunho na mesa" — veja a §3).
- **Testes** = a **fiscalização de qualidade** em cada etapa (veja a §5).

1 Regras de ouro do Copilot

 **Analogia da obra:** o GitHub Copilot é como um **ajudante experiente na obra**: rápido e cheio de repertório, mas quem assina a planta é você (o engenheiro). As "regras de ouro" são como dar ordens claras ao ajudante — peça uma parede de cada vez, confira o prumo antes de seguir, e nunca aceite um serviço que você não entende.

- **Dê contexto.** Deixe abertos os arquivos relevantes e use `@workspace` no Copilot Chat.
- **Peça em partes pequenas.** Um endpoint, um componente, um teste por vez.
- **Diga a stack e as regras.** "Node + Express", "status do projeto", "só o dono exclui".
- **Peça os testes junto.** Toda função/rota nova vem com teste.
- **Leia antes de aceitar.** Você assina o código. Não entendeu? `/explain` .
- **Teste e versione a cada passo.** Rode a suíte e faça `git commit` quando algo funcionar.

- **Seletores estáveis para E2E.** Ao pedir componentes, peça `data-testid` nos botões de ação (candidatar, selecionar, excluir, enviar-avaliação, **sair**). É o que faz o robô do Playwright não quebrar quando o texto/visual muda.
- **Cubra os dois lados de cada regra.** Fez login? Teste o logout. Salvou no `localStorage` ? Teste que ele é limpo. Cobertura alta nasce de testar entradas e saídas.

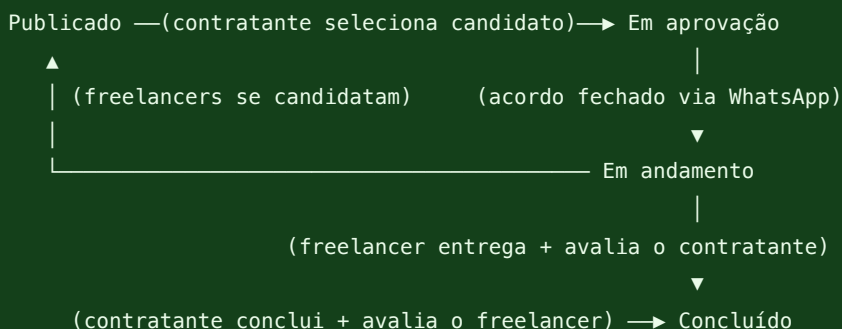
2 O app: FreelaAvalia 360 (recrutamento 360°)

Analogia da obra: as **entidades** (Usuário, Projeto, Candidatura, Avaliação) são os **cômodos e registros do prédio** — cada um guarda um tipo de coisa. Já a **máquina de status** é o **cronograma da obra**: um projeto começa na "fundação" (Publicado), sobe a "estrutura" (Em aprovação), entra no "acabamento" (Em andamento) e recebe o "habite-se" (Concluído). Não dá para pintar a parede antes de levá-la — as etapas seguem uma ordem.

Contratantes publicam projetos abertos; freelancers se candidatam; o contratante vê os candidatos com a reputação de cada um e seleciona; ao final, os dois se avaliam. A avaliação mútua (1 a 5 + comentário) é a **avaliação 360°**.

Entidade	O que guarda	Regras principais
Usuário	nome, e-mail, papel (freelancer/contratante), senha, telefone e endereço (opcionais)	e-mail único; papel válido; senha ≥ 4 ; perfil editável; senha nunca sai nas respostas
Projeto	título, descrição, contratante, freelancer (opcional), contato, status	nasce aberto; percorre a máquina de status; só o dono edita/exclui
Candidatura	projeto + freelancer inscrito	1 por freelancer por projeto; só em projeto aberto
Avaliação	de quem, para quem, nota (1–5), comentário	durante o andamento ou após concluir; 1 por lado

A máquina de status do projeto



Regras do MVP: projeto em andamento não pode ser editado nem reaberto — para mudar, publique um novo. Editar/Excluir só aparecem enquanto o projeto está **Publicado**. A negociação de valores acontece no WhatsApp, fora do app; o contratante apenas registra no sistema quando fecha o acordo.

3 Estrutura de dados: como guardamos usuários e cenários "no localhost"

 **Analogia da obra:** hoje os dados ficam **em memória** — como **materiais espalhados no chão da obra**: prático para trabalhar, mas se der uma chuva (reiniciar o servidor) tudo some. Já o **localStorage** do navegador é o **crachá no bolso do trabalhador**: guarda só quem ele é (a sessão), para não precisar se identificar toda hora na portaria. O **seed** (`USUARIOS_PADRAO`) é o **kit inicial de materiais** que já deixamos no canteiro para a aula começar sem montar tudo à mão.

Este é um ponto importante para entender o projeto — e para saber o que muda quando ele crescer. **Hoje não existe banco de dados.** Os dados vivem em dois lugares diferentes, os dois "no localhost":

3.1 Backend — arrays em memória (`repositorio.js`)

Todo o estado do sistema fica em quatro listas simples dentro do processo Node, com um contador de ids:

```
let usuarios      = []; // { id, nome, email, papel, senha, endereco, telefone }
let contratos     = []; // { id, titulo, descricao, contratanteId, freelancerId, email, endereco, telef
let avaliacoes    = []; // { id, contratoId, deId, paraId, nota, comentario }
let candidaturas = []; // { id, contratoId, freelancerId }
let seq = 1;          // gerador de ids incremental
```

Como isso funciona na prática:

- **Relacionamentos por id (chave estrangeira "manual").** Um contrato guarda `contratanteId` e `freelancerId`; uma candidatura guarda `contratoId` + `freelancerId`; uma avaliação guarda `deId` / `paraId`. Não há `JOIN` — o código faz `array.find` / `array.filter` para cruzar as listas (ex.: `avaliacoesRecebidas(paraId)`).
- **As "regras de integridade" moram no código, não no banco.** "E-mail único", "1 candidatura por freelancer", "1 avaliação por lado" e a máquina de status são garantidas por funções como `acharUsuarioPorEmail`, `jaCandidatou` e `jaAvaliou` — não por constraints de um banco.
- **Volátil.** Ao reiniciar o backend, **tudo se perde**. Para a aula isso é ótimo (estado limpo), mas não serve para produção.
- **Dados semeados (`USUARIOS_PADRAO` + `semearUsuarios`).** Para não começar do zero a cada `npm start`, o servidor cria automaticamente um contratante e dois freelancers (senha `1234`). É o que permite os cenários de aula — como **duas freelancers disputando a mesma vaga** — sem precisar cadastrar ninguém à mão.
- **`reset()` = isolamento de teste.** Antes de cada teste de integração, `reset()` zera as quatro listas e o `seq`. É por isso que os 64 testes de backend rodam em menos de 1s: sem I/O, sem banco.

 **Cenários "prontos" = seed.** Quando você ouve "cenário em localhost", pense em duas peças: os **dados semeados** (`USUARIOS_PADRAO`) que criam o palco, e o **estado em memória** que evolui durante a sessão (projetos publicados, candidaturas, avaliações). Reiniciou → volta ao palco inicial.

3.2 Frontend — `localStorage` para a sessão do usuário (`App.jsx`)

O frontend **não** guarda projetos nem avaliações — isso é sempre buscado da API. O `localStorage` guarda **uma única coisa**: quem está logado.

```
const CHAVE = "freelavalia_user";
// login → localStorage.setItem(CHAVE, JSON.stringify(user))
// logout → localStorage.removeItem(CHAVE)
// abrir → JSON.parse(localStorage.getItem(CHAVE)) // restaura a sessão
```

- **Por que `localStorage` ?** Para a sessão sobreviver ao `*refresh*` da página. Ao abrir o app, `carregarUser()` lê a chave e já entra logado.
- **Logout é a operação inversa do login.** `sair()` faz `removeItem` e volta para a Landing. É exatamente isso que o novo teste garante: depois do logout, a chave some e um `*reload*` **não** reloga sozinho.
- **Não é seguro para dados sensíveis.** A senha nunca é gravada (o backend nunca a devolve). Em produção, o ideal é um token de sessão (JWT) com expiração — não o objeto do usuário inteiro.

Camada	Onde vive	O que guarda	Persiste ao reiniciar?
Backend	Arrays em memória (<code>repositorio.js</code>)	Usuários, projetos, candidaturas, avaliações	❌ Não (volátil + seed no start)
Frontend	<code>localStorage</code> do navegador	Só o usuário logado (sessão)	✅ Sim, até fazer logout

4 Próximos passos: migrar para um banco de dados

 **Analogia da obra:** trocar a memória por um **banco de dados** é sair dos "materiais no chão" para um **cartório de registros**, onde tudo fica arquivado com segurança e permanência. O **banco relacional** (PostgreSQL) é como **arquivos com fichas que se referenciam** (esta obra pertence a este cliente), com regras rígidas de integridade — o próprio cartório recusa uma ficha órfã. O **não relacional** (Firestore) é como **caixas flexíveis**: rápido de usar e sem obra de infraestrutura, mas você mesmo precisa garantir a organização. O **Prisma** é o **encarregado do arquivo** que fala com o cartório por você.

O armazenamento em memória cumpre bem o papel didático. O passo natural é dar **persistência real**. Recomendação, em ordem:

4.1 Recomendação principal — banco RELACIONAL (PostgreSQL + Prisma)

O domínio é **fortemente relacional**: usuários, projetos, candidaturas e avaliações têm chaves e regras de integridade claras. Um banco relacional resolve no próprio schema o que hoje é código:

- `email UNIQUE` no usuário (substitui `acharUsuarioPorEmail`);
- `UNIQUE(contratoId, freelancerId)` na candidatura (substitui `jaCandidatou`);
- `UNIQUE(contratoId, deId)` na avaliação (substitui `jaAvaliou`);
- `FOREIGN KEY` em `contratanteId` , `freelancerId` , `contratoId` (garante que não sobra "id órfão");
- `status` com `CHECK /enum` para a máquina de estados.

Com **Prisma** (ORM para Node), a migração é suave porque a fronteira já está isolada: **só o `repositorio.js` muda**. `app.js` e todos os testes de integração continuam iguais — troca-se o "como guarda", não o "o que faz".

```

model Usuario {
  id          Int      @id @default(autoincrement())
  nome        String
  email       String   @unique
  papel       String
  senha       String
  telefone    String?
  endereco    String?
  projetos    Contrato[] @relation("Contratante")
  candidaturas Candidatura[]
}

model Contrato {
  id          Int      @id @default(autoincrement())
  titulo      String
  descricao    String?
  status      String   @default("aberto")
  contratanteId Int
  freelancerId Int?
  candidaturas Candidatura[]
  avaliacoes  Avaliacao[]
}

model Candidatura {
  id          Int @id @default(autoincrement())
  contratoId  Int
  freelancerId Int
  @@unique([contratoId, freelancerId]) // "1 candidatura por freelancer"
}

model Avaliacao {
  id          Int      @id @default(autoincrement())
  contratoId  Int
  deId        Int
  paraId      Int
  nota        Int
  comentario  String?
  @@unique([contratoId, deId])          // "1 avaliação por lado"
}

```

4.2 Alternativa pragmática — NÃO relacional (Firestore)

O repositório **já usa Firebase** (a landing do Projeto Garapuvu está hospedada lá: `firebase.js`, `firestore.rules`). Isso torna o **Firestore** (NoSQL, orientado a documentos) a opção de menor atrito:

- Sem servidor de banco para manter; escala e backup automáticos; SDK no próprio front.
- Coleções `usuarios`, `contratos`, `candidaturas`, `avaliacoes`.
- **Custo:** as regras de unicidade e integridade voltam a ser responsabilidade do código/ `firestore.rules` (não há `UNIQUE` / `FOREIGN KEY` nativos), e consultas com muitos cruzamentos ficam menos naturais que um `JOIN`.

4.3 "Os dois" — quando faz sentido

Uma arquitetura madura pode combinar: **PostgreSQL** para o núcleo transacional (contas, contratos, avaliações — onde integridade e consistência são inegociáveis) e **Firestore/Redis** para o que é volátil ou em tempo real (notificações, presença, chat, contadores de visita). Para o estágio atual do projeto, isso é **over-engineering**: comece com **um** banco.

💡 **Sugestão de rota:** comece com **PostgreSQL + Prisma** (integridade no schema, migração isolada ao `repositorio.js`, testes de integração intactos). Adote Firestore só se a prioridade for "zero infra" aproveitando o Firebase que você já tem.

PROMPT 4.A — MIGRAR O REPOSITÓRIO PARA PRISMA (sem quebrar a API)

@workspace Quero persistir os dados em PostgreSQL usando Prisma, mantendo a API e os testes de integração intactos. Crie o `schema.prisma` com `Usuario`, `Contrato`, `Candidatura` e `Avaliacao` (com `@unique` em `email`, `[contratoId,freelancerId]` e `[contratoId,deId]`). Reescreva SOMENTE o `repositorio.js` para usar o Prisma Client, preservando as MESMAS funções exportadas (`criarUsuario`, `acharUsuarioPorEmail`, `jaCandidatou`, `avaliacoesRecebidas`, `reset...`) para que `app.js` e `app.test.js` não mudem. Faça o `reset()` truncar as tabelas (uso em teste).

5 A pirâmide de testes e a cobertura

🏠 **Analogia da obra:** os testes são a **fiscalização de qualidade**, em camadas:

- **Unitário** = testar **um tijolo/uma peça** isolada (esta viga aguenta o peso?). Rápido e barato — é a **base da pirâmide**.
- **Integração** = testar **uma parede montada** (os tijolos + argamassa funcionam juntos?).
- **Componente/E2E** = **percorrer a casa pronta** como um morador faria (a porta abre, a luz acende, a torneira funciona).
- **Cobertura** = a **porcentagem da obra que o fiscal realmente inspecionou**. 100% quer dizer que ele passou por cada cômodo — mas cuidado: inspecionar tudo não garante que a casa seja bonita, só que foi verificada.

Nível	No FreelaAvalia 360	Arquivo	Qtd
Unitário	validar nota, média, e-mail, <code>podeAvaliar</code> ; repositório em memória; <code>semSenha</code>	<code>regras.test.js</code> , <code>repositorio.test.js</code> , <code>app.test.js</code>	27
Integração	usuários/login/logout, CRUD, recrutamento, avaliação 360, bordas/exceções	<code>app.test.js</code>	37
Componente/Unidade (front)	<code>Estrelas</code> , <code>Modal</code> , <code>Landing</code> , <code>Auth</code> , <code>Avaliar</code> , <code>Perfil</code> , <code>Projetos</code> (+ sub- componentes de ação), <code>App</code> (logout), <code>api.js</code>	<code>*.test.jsx</code> , <code>api.test.js</code>	74
E2E	logout, CRUD, fluxo completo (2 atores), 2 vagas, seleção com remoção, avaliação	<code>frontend/e2e/*.spec.js</code>	6

Cobertura de código (provider `v8`) — 100%

```
make coverage    # gera os dois relatórios de uma vez
# ou, individualmente:
cd backend && npm run coverage    # relatório em backend/coverage/index.html
cd frontend && npm run coverage    # relatório em frontend/coverage/index.html
```

- **Backend:** 100% em statements, branches, functions e lines (`regras.js` , `repositorio.js` , `app.js`).

- **Frontend: 100%** em statements, branches, functions e lines, com threshold de **60%** no `vite.config.js` (o build falha se cair abaixo). A cobertura vem de testes de componente (RTL) para cada página e para a camada `api.js`.

🕒 **Cuidado com a métrica.** O coverage do `vitest` **não enxerga** o que os testes E2E do Playwright exercitam. Por isso as páginas precisam de testes de unidade (RTL) próprios para aparecerem na cobertura — E2E valida o fluxo, unidade valida (e mede) as ramificações.

🎯 **Como chegamos a 100% sem "testar mentira".** Duas lições que valem mais que o número:

1. **Exceções alcançáveis viram teste** — corpo ausente, ids inexistentes, falhas de API, seleção em projeto fechado. São testes reais de robustez (`describe "Bordas e exceções"` no backend; caminhos de erro no front).
2. **Código morto não se testa — se elimina ou se justifica.** Os guardas `req.body || {}` eram redundantes (o `express.json()` já garante `req.body = {}`), então foram **removidos** — é o tratamento correto de código morto. Já um sub-componente de UI difícil de alcançar pela tela (`AcoesFreelancer`) foi **exportado e testado como unidade**, em vez de forçar um caminho artificial.

PROMPT 5.1 — TESTAR O LOGOUT (E2E + UNIDADE)

```
@workspace Crie frontend/e2e/logout.spec.js: cadastra/entra, confirma que o app aparece e que o localStorage tem a chave freelavalua_user, clica em data-testid="btn-sair", verifica que voltou para a Landing, que a chave foi removida e que um reload NÃO reloga sozinho; por fim, entra de novo pela tela de login. Some um teste de unidade em src/App.test.jsx mockando as páginas (Landing/Auth/Projetos/Perfil) para validar login (grava no localStorage), logout (limpa) e a restauração da sessão ao montar.
```

PROMPT 5.2 — SUBIR A COBERTURA DO FRONTEND ACIMA DE 60%

```
@workspace Escreva testes de componente (React Testing Library) mockando ../api.js para: Auth (troca de abas, login/cadastro chamando a API, exibição de erro, voltar), Avaliar (botão desabilitado com nota 0, envio chamando api.avaliar + aoConcluir, erro), Perfil (carrega reputação, salva e chama aoAtualizar, erro), Landing e Modal (callbacks e stopPropagation), Projetos (ações por papel/status do contratante e do freelancer, formulário criar/editar, modal de candidatos com selecionar/remover) e api.js (mockando fetch: sucesso, erro com mensagem da API, status genérico e falha de rede). Meta: cobertura > 60% no vite.config.js.
```

5.3 Preciso de mock? Sim — no frontend. Não — no backend

🔧 **Analogia da obra:** um **mock** é um **dublê / peça de mentira** usada só no teste. Para testar se uma **fechadura** funciona, você a monta num **batente de teste** na bancada — não precisa construir a casa inteira em volta. O mock é esse batente: substitui o que está **na borda** (a rede, o banco) para você testar a peça isolada. A regra: dublê só no que está em volta, **nunca** na peça que você está inspecionando.

Resposta curta: sim, foi necessário mockar — mas só no frontend, e por um bom motivo. O padrão que usamos foi **mockar a fronteira** (a borda que sai do código sob teste), nunca a lógica que queremos validar.

Backend — ZERO mocks. Os testes de API usam **Supertest** chamando a app Express de verdade, sobre o repositório em memória real (`reset()` isola cada teste). Como não há banco nem rede, não há nada para mockar — o teste exercita o código real de ponta a ponta, o que dá mais confiança. Os testes unitários (`regras.js` , `repositorio.js` , `semSenha`) também são funções puras: sem mock.

Frontend — mocks na fronteira. Aqui o mock é necessário porque os componentes conversam com o mundo externo:

O que foi mockado	Onde	Por quê
<code>../api.js</code> (<code>vi.mock</code>)	<code>Auth</code> , <code>Avaliar</code> , <code>Perfil</code> , <code>Projetos</code> , <code>App</code>	Isola o componente da rede/HTTP . Testamos a UI (o que aparece, o que é chamado) sem depender de um backend no ar. Permite simular sucesso e erro (<code>mockRejectedValue</code>).
<code>global.fetch</code>	<code>api.test.js</code>	Para testar a própria <code>api.js</code> (tradução de erros), mockamos o <code>fetch</code> — a fronteira real dela.
As páginas (<code>Landing</code> , <code>Auth</code> , <code>Projetos</code> , <code>Perfil</code>)	<code>App.test.jsx</code>	Para testar só a lógica de sessão do <code>App</code> (login/logout/localStorage) sem arrastar as páginas inteiras.
<code>localStorage</code> , <code>window.confirm</code>	vários	APIs do navegador simuladas pelo <code>jsdom</code> / <code>vi.spyOn</code> .

🧠 **Regra de ouro dos mocks:** mocke o que está **na borda** do que você testa (rede, relógio, navegador) — **nunca** a regra de negócio em si. Se você se pega mockando aquilo que o teste deveria verificar, o teste vira decoração. E lembre: **o E2E não usa nenhum mock** — ele sobe backend + frontend reais e é a prova final de que as peças conversam.

6 O Makefile: um comando para cada coisa

🔧 **Analogia da obra:** o `Makefile` é o **painel de comando do mestre de obras**: em vez de gritar dez instruções, ele aperta um botão. `make dev` = "liga a obra"; `make test` = "chama a fiscalização rápida"; `make coverage` = "emite o laudo de inspeção"; `make stop` = "encerra o expediente".

```
make install    # instala dependências (backend + frontend)
make dev        # sobe backend :3001 + frontend :5173 juntos
make test       # testes unitários/integração dos dois (vitest) — rápido, sem browser
make test-e2e   # testes E2E do Playwright (browser) — sobe os serviços sozinho
make test-all  # tudo: vitest + Playwright
make coverage   # gera os relatórios de cobertura (backend/ e frontend/coverage/index.html)
make stop       # encerra processos nas portas 3001 e 5173-5175
```

✓ **Por que separar `test` de `test-e2e` ?** `make test` é a suíte rápida do dia a dia (rodou em <2s). `make test-e2e` abre o navegador e é mais lento. `make test-all` é o "selo verde" antes de um commit importante. `make coverage` gera os dois relatórios HTML de uma vez.

PROMPT 6.1 — ADICIONAR OS ALVOS DE E2E E COBERTURA AO MAKEFILE

@workspace No `Makefile`, adicione os alvos: `test-e2e` (`cd frontend && npm run e2e`), `test-all` (depende de `test` e `test-e2e`) e `coverage` (roda `npm run coverage` no backend e no frontend). Atualize o `.PHONY` e o "`make help`" para listá-los, e o comentário do topo.

7 Smoke test E2E com dois atores

 **Analogia da obra:** o smoke test com dois atores é o **ensaio geral com moradores de verdade** antes da entrega: um contratante e um freelancer usam o prédio ao mesmo tempo, em janelas separadas, e a gente confere se o ciclo completo funciona — da publicação da vaga à avaliação final. É a prova de que **as peças conversam** na vida real, sem dublês.

O smoke test percorre o caminho crítico de ponta a ponta usando dois contextos de navegador (contratante e freelancer logados ao mesmo tempo) e pode rodar em modo **headed** com **slowMo** alto, para dar pra acompanhar cada interação na tela durante a aula.

```
cd frontend
npm run e2e                # todos os E2E (headless) – sobe os serviços sozinho
npm run e2e:smoke          # fluxo completo, headed + slowMo
npm run e2e:smoke:selecao  # seleção com remoção de candidato
npm run e2e:smoke:duas-vagas # uma freelancer conclui duas vagas
```

Como o webServer ajuda: `playwright.config.js` sobe e aguarda backend (`:3001`) e frontend (`:5173`) sozinho, com `reuseExistingServer` para reaproveitar o que já estiver no ar.

8 Tags e scripts por tipo de teste

 **Analogia da obra:** as **tags** (`@unitario` , `@integracao` , `@e2e`) são as **etiquetas nos laudos de fiscalização**: permitem pedir só um tipo de inspeção ("me traga só os laudos elétricos") sem revirar a papelada toda.

Cada teste recebe uma **tag** no título, para `grep` nos arquivos e para filtrar a execução (`vitest -t / playwright --grep`).

Tag	Tipo de teste	Onde vive
<code>@unitario</code>	regras de negócio	<code>backend/src/regras.test.js</code>
<code>@integracao</code>	API (Supertest)	<code>backend/src/app.test.js</code>
<code>@interface</code>	componente/unidade (RTL) + E2E	<code>src/**/*.test.jsx</code> , specs E2E
<code>@e2e</code>	só os E2E de navegador	<code>frontend/e2e/*.spec.js</code>

```
# backend
npm run test:unit      # só regras de negócio
npm run test:integracao # só a API
# frontend
npm run test:interface # só componente/unidade
npm run e2e:e2e        # só E2E de navegador
```

9

Endpoints da API

 **Analogia da obra:** cada **endpoint** é uma **janelinha de atendimento** na portaria do prédio, e cada uma faz uma coisa só. O **método** é o tipo de pedido: **POST** = "cadastrar algo novo" (abrir ficha), **GET** = "consultar" (pedir informação), **PATCH** = "atualizar um dado", **DELETE** = "remover". A **rota** (`/contratos/:id`) é o **endereço** daquela janelinha.

Método	Rota	Descrição
POST	<code>/usuarios</code>	Cadastro (endereço/telefone opcionais)
POST	<code>/login</code>	Autentica (retorna o usuário sem a senha)
POST	<code>/logout</code>	Encerra a sessão (stateless: confirma o logout de um usuário conhecido)
GET	<code>/usuarios/:id</code>	Usuário + média das avaliações (404 se não existe)
PATCH	<code>/usuarios/:id</code>	Edita o próprio perfil
POST	<code>/contratos</code>	Publica um projeto aberto
GET	<code>/contratos</code>	Lista projetos (com candidatos e avaliadores)
PATCH	<code>/contratos/:id</code>	Edita título/descrição/contato
DELETE	<code>/contratos/:id</code>	Exclui (só publicado e sem candidatos)
POST	<code>/contratos/:id/candidaturas</code>	Freelancer se candidata
GET	<code>/contratos/:id/candidaturas</code>	Candidatos com reputação
DELETE	<code>/contratos/:id/candidaturas/:fid</code>	Retira candidatura
PATCH	<code>/contratos/:id/selecionar</code>	Seleciona candidato → Em aprovação
PATCH	<code>/contratos/:id/andamento</code>	Confirma acordo → Em andamento
PATCH	<code>/contratos/:id/concluir</code>	Encerra → Concluído
POST	<code>/avaliacoes</code>	Avaliação 360 (em andamento/concluído)
GET	<code>/avaliacoes/usuario/:id</code>	Avaliações recebidas por um usuário

10

Conceitos-chave do projeto

 **Analogia da obra:** esta é a **legenda da planta** — os símbolos e termos que aparecem no projeto todo, reunidos num só lugar para consulta rápida.

Conceito	O que é	Onde aparece
Máquina de estados	Status que só avançam numa ordem válida	<code>app.js</code> (selecionar/andamento/concluir)
Repositório isolado	Trocar o "como guarda" sem mexer na API	<code>repositorio.js</code> (memória hoje, banco amanhã)
Sessão no cliente	Login/logout persistidos no <code>localStorage</code>	<code>App.jsx</code> (<code>login</code> / <code>sair</code>)
Regras de autorização	Quem pode excluir/editar e quando	DELETE guard + botões por papel
Seletores estáveis	<code>data-testid</code> resistente a mudança de visual	botões de ação
Cobertura de código	Medir quais ramos os testes exercitam	<code>coverage/</code> + <code>threshold</code> no <code>vite.config.js</code>
Multi-ator no E2E	Dois contextos de navegador no mesmo teste	<code>fluxo-completo.spec.js</code>
Orquestração via Make	Um comando por tarefa	<code>Makefile</code> (<code>test</code> , <code>test-e2e</code> , <code>test-all</code>)

Para apresentar aos alunos

Rode a **pirâmide viva**: unitários (instantâneos) → integração (64 testes de backend) → componente/unidade com **cobertura de 100%** (`make coverage` , abra o `coverage/index.html`) → e, por fim, `npm run e2e:smoke` : o navegador abre e o projeto percorre todo o ciclo em duas janelas, incluindo o **logout** ao final. Depois mostre o `repositorio.js` e conte a história: "hoje é memória; amanhã, um `UNIQUE` no banco faz esse `if` sumir". É o melhor "final de aula": o sistema inteiro funcionando — e o caminho de evolução visível.